

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA4301

COURSE OUTCOME COVERED

CO 5: Construct interoperable web services by leveraging JAX-RPC, WSDL, SOAP, and XML Schema for seamless data exchange across distributed systems.(L4,L5)

Assessment Tool : Industry Problem Task

Weightage: 40%

QUESTIONS:

Total Marks: 30

1.A hospital management system requires a **SOAP-based web service** to manage patient information such as registration, appointment scheduling, and medical records. The system must ensure secure, structured, and interoperable communication between different hospital departments.

Design a **SOAP-based web service** for the hospital system. Explain the structure of SOAP messages (Envelope, Header, Body), define the operations involved (e.g., add patient, get records), and illustrate how data is exchanged between client and server. Justify the use of SOAP in this scenario considering reliability and security.

2.An enterprise application needs to expose its web services so that external systems can understand how to interact with them. The organization plans to use **WSDL (Web Services Description Language)** to describe the service.

Design the **WSDL structure** for a web service that provides operations such as retrieving and updating data. Explain the key components of WSDL (types, message, portType, binding, service), and illustrate how they define the service interface and communication details. Provide a clear structure and justify its importance in service integration.

3. A company is experiencing issues in its SOAP-based communication system where requests are not processed correctly, and responses are either delayed or incorrect.

Analyze the SOAP communication process and identify possible issues such as message formatting errors, incorrect namespaces, or endpoint problems. Propose a debugging approach to resolve these issues, including validation of SOAP messages and error handling techniques. Explain how the corrected system ensures reliable communication.

Code of Conduct:

I __CH.Bhavya Sri/192411249_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

CH.Bhavya Sri

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis	Good analysis	Basic analysis	Poor or no

		with validated results and performance evaluation	with reasonable validation	with limited validation	analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

1. SOAP-Based Web Service for Hospital Management System

Introduction

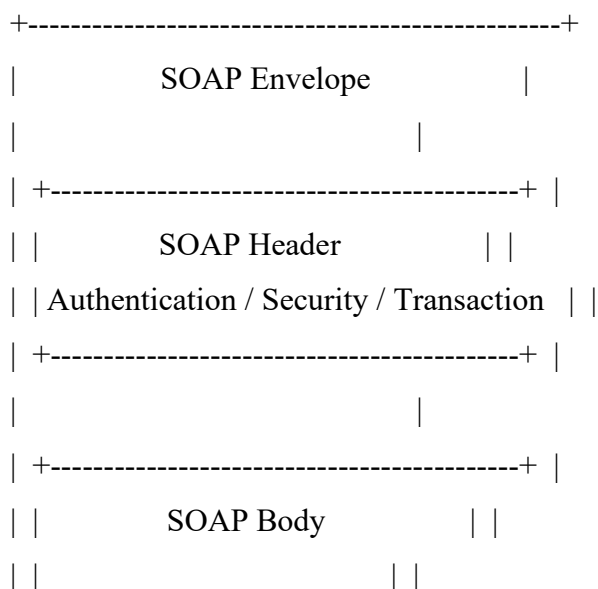
A **SOAP (Simple Object Access Protocol)** web service provides a standardized way for hospital departments to exchange structured patient information. It is suitable when the system requires strong contracts, standardized error handling, security extensions, and interoperability between different platforms.

Main Operations

Operation	Purpose	Input	Output
addPatient	Register a new patient	Patient details	Patient ID/status
getPatient	Retrieve patient details	Patient ID	Patient information
scheduleAppointment	Schedule an appointment	Patient ID, doctor, date	Appointment status
getMedicalRecords	Retrieve medical history	Patient ID	Medical records
updatePatient	Update patient information	Patient details	Update status

SOAP Message Structure

A SOAP message consists mainly of an **Envelope**, optional **Header**, and **Body**.



```

| |   addPatient / getRecords / etc.   | |
| |                                   | |
| +-----+
+-----+

```

Example: Add Patient Request

```

<soap:Envelope
  xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:hos="http://hospital.example.com/">

  <soap:Header>
    <hos:Authentication>
      <hos:Token>ABC123</hos:Token>
    </hos:Authentication>
  </soap:Header>

  <soap:Body>
    <hos:addPatient>
      <hos:name>Ravi Kumar</hos:name>
      <hos:age>45</hos:age>
      <hos:gender>Male</hos:gender>
      <hos:department>Cardiology</hos:department>
    </hos:addPatient>
  </soap:Body>

</soap:Envelope>

```

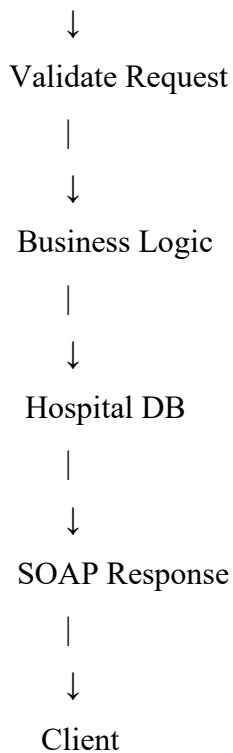
Data Exchange

Hospital Department / Client

```

|
| SOAP Request
↓
SOAP Web Service
|

```



Example Response

```
<soap:Envelope
  xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">

  <soap:Body>
    <addPatientResponse>
      <patientId>P10025</patientId>
      <status>Patient Registered Successfully</status>
    </addPatientResponse>
  </soap:Body>

</soap:Envelope>
```

Why SOAP is Suitable

Interoperability: Different hospital departments can communicate even when they use different programming languages or platforms.

Structured communication: XML provides a standardized message format.

Security: SOAP can use **WS-Security** for message-level security such as authentication and digital signatures.

Reliability: SOAP supports standardized fault messages and can be used with enterprise messaging and reliability mechanisms.

Formal contract: WSDL can clearly define operations, messages, data types and communication details.

Error handling: SOAP Fault provides a standard mechanism for reporting processing errors.

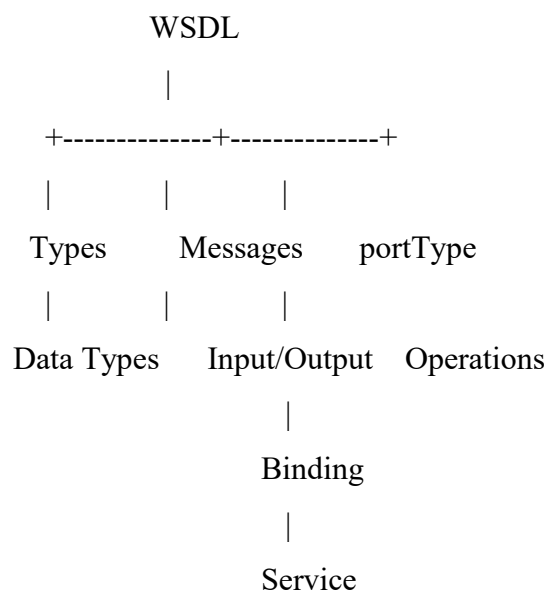
Conclusion: SOAP is appropriate for hospital systems where structured communication, interoperability, security, formal service contracts and reliable error handling are important.

2. WSDL Structure for an Enterprise Web Service

Introduction

WSDL (Web Services Description Language) is an XML-based language used to describe a web service. It tells clients **what operations are available, what data they require, what they return, how communication takes place, and where the service is located.**

Main Components of WSDL



|
Endpoint

1. Types

Defines the data types used by the service, commonly using **XML Schema (XSD)**.

Example:

```
<types>
  <xsd:schema>
    <xsd:element name="customerId"
      type="xsd:int"/>
    <xsd:element name="customerName"
      type="xsd:string"/>
  </xsd:schema>
</types>
```

2. Message

Defines the data exchanged between client and server.

```
<message name="GetDataRequest">
  <part name="id" type="xsd:int"/>
</message>

<message name="GetDataResponse">
  <part name="data" type="xsd:string"/>
</message>
```

3. portType

Defines the **operations** provided by the service and the messages associated with them.

```
<portType name="DataServicePortType">
```



```

<operation name="getData">
  <input message="tns:GetDataRequest"/>
  <output message="tns:GetDataResponse"/>
</operation>

<operation name="updateData">
  <input message="tns:UpdateDataRequest"/>
  <output message="tns:UpdateDataResponse"/>
</operation>

</portType>

```

4. Binding

Specifies how the operations are communicated. It can define the SOAP version, communication style and transport protocol.

```

<binding name="DataServiceBinding"
  type="tns:DataServicePortType">

  <soap:binding
    transport="http://schemas.xmlsoap.org/soap/http"
    style="document"/>

</binding>

```

5. Service

Defines the actual service endpoint where the client can access the web service.

```

<service name="DataService">

  <port name="DataServicePort"
    binding="tns:DataServiceBinding">

```

```
<soap:address  
  location="https://example.com/DataService"/>
```

```
</port>
```

```
</service>
```

Overall WSDL Structure

WSDL Document

```
|  
+-- Types  
|   └── Data definitions  
|  
+-- Messages  
|   └── Input / Output  
|  
+-- portType  
|   └── Operations  
|  
+-- Binding  
|   └── SOAP + HTTP details  
|  
+-- Service  
    └── Endpoint address
```

Importance in Service Integration

WSDL acts as a **formal contract between service providers and clients**. A client can use the WSDL to understand the available operations, required input, expected output, communication protocol and service endpoint without knowing how the service is internally implemented.

Conclusion: WSDL simplifies integration by providing a standardized machine-readable description of a web service.

3. Debugging SOAP Communication Problems

Introduction

When SOAP requests fail, the problem may occur at different stages, including message creation, XML validation, namespace processing, network communication, server processing, or response generation.

Common Problems

Problem	Possible Cause
Invalid SOAP request	Incorrect XML structure
Namespace error	Wrong or missing namespace URI
Operation not recognized	Incorrect operation name
Request rejected	Invalid authentication/security information
Endpoint error	Incorrect service URL
HTTP error	Wrong HTTP method, headers or server configuration
Delayed response	Network/server/database performance issue
Incorrect response	Server-side business logic error
SOAP Fault	Service detected an application or protocol error

SOAP Debugging Process

Client Request



Check Endpoint



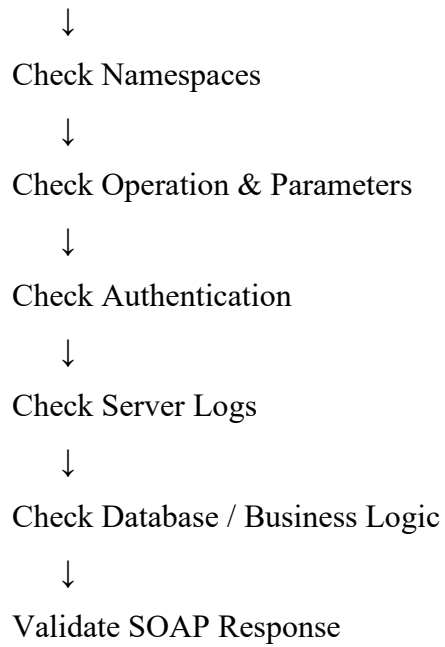
Check HTTP Headers



Validate XML Structure



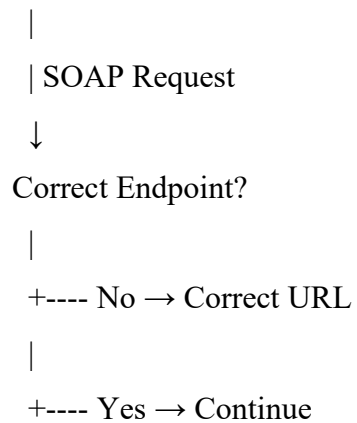
Check SOAP Envelope



Step 1: Check the Endpoint

Verify that the client is sending the request to the correct service URL.

Client



Step 2: Validate SOAP Message

Check whether the message contains the required:

SOAP Envelope

Header, if required

Body

Operation

Parameters

Example:

```
<soap:Envelope>
  <soap:Header>
    ...
  </soap:Header>

  <soap:Body>
    <getPatient>
      <patientId>P10025</patientId>
    </getPatient>
  </soap:Body>
</soap:Envelope>
```

Step 3: Check Namespaces

Incorrect namespaces can cause the server to fail to recognize an operation.

```
<soap:Envelope
  xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:hos="http://hospital.example.com/">
```

The namespace used by the client must match the namespace expected by the service contract.

Step 4: Validate Against WSDL

Compare the request with the WSDL to verify:

Operation name

Input parameters

Data types

Message structure

Binding

Endpoint

Step 5: Check SOAP Fault

SOAP provides a standard **Fault** structure for reporting errors.

```
<soap:Fault>  
  <faultcode>soap:Client</faultcode>  
  <faultstring>Invalid Patient ID</faultstring>  
</soap:Fault>
```

The client should read the fault and display or log an appropriate error instead of treating the response as successful.

Step 6: Check Server Logs

If the request reaches the server but produces an incorrect response, inspect:

Application logs

Authentication/security logs

Database logs

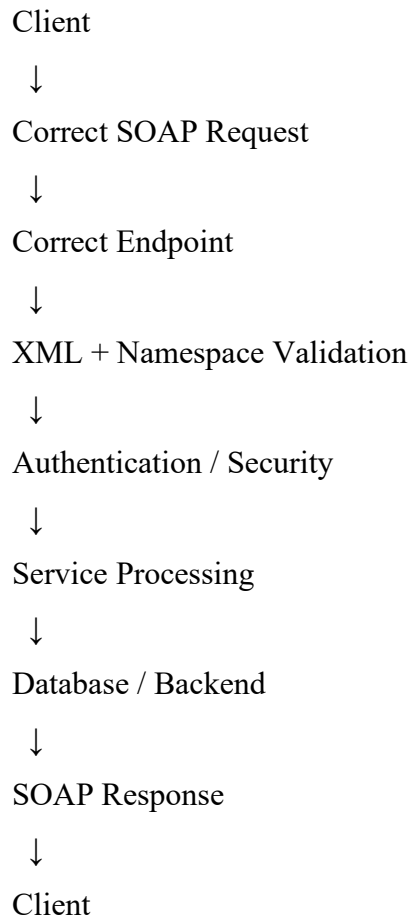
Server exceptions

Request and response timestamps

Step 7: Test Request and Response

Use a SOAP testing tool or equivalent HTTP client to send a known valid request. Compare the actual response with the expected WSDL-defined response.

Corrected Communication



How Reliability is Improved

The corrected system becomes more reliable by:

Validating SOAP messages before processing.

Using the correct namespaces and WSDL definitions.

Verifying the endpoint and HTTP configuration.

Implementing proper authentication and security.

Handling SOAP Faults correctly.

Maintaining server-side logs for troubleshooting.

Using appropriate timeout and retry mechanisms where supported.

Validating the response before displaying the result.

Conclusion: SOAP communication problems can be systematically diagnosed by checking the endpoint, HTTP configuration, XML structure, namespaces, WSDL contract, security, server processing and SOAP Faults. Proper validation and standardized error handling make communication more reliable and easier to maintain.